12. API Contract Specification

Service contracts for experience, decision, validation, and admin workflows

Project: QuantPipeline

Document status: Working baseline for product and engineering delivery

Version: v1.0

Date: 2026-04-23

Audience: Product owner, architecture, development, QA, and future implementation agents

Purpose: This document is intended to be implementable. It defines contracts, boundaries, and decisions that should survive future feature growth without breaking the core recommendation pipeline.

Document Control

Field	Value
Document code	12
Title	API Contract Specification
Primary input set	Documents 01-09, A1 technology recommendation, and baseline truth table
Primary architecture anchor	Data & Feature Layer -> Signals -> Selection -> Allocation -> Timing -> Risk Overlay -> Recommendation Object -> Backtest / Paper / Replay / Ops
Assumed release posture	Private single-user first release with future-safe modular boundaries
Key dependencies	Documents 10 and 11, 08 PRD, 09 Functional Requirements
Quality rule	No silent degradation, explicit trust semantics, and replayable output

1. API Contract Purpose

This document defines the service contract surface of QuantPipeline. It covers the request and response expectations for the experience layer, scenario workflows, replay, validation, paper monitoring, and admin operations. The goal is to make the platform buildable across web, mobile web, and future native clients without changing the meaning of core domain objects.

The API surface is intentionally recommendation-centric and audit-aware. Endpoints should not expose backend implementation fragments without user meaning. They should deliver objects that map to real screens, decision questions, and operator workflows while still carrying trace identifiers and stage references for debugging and replay.

2. API Surface Map

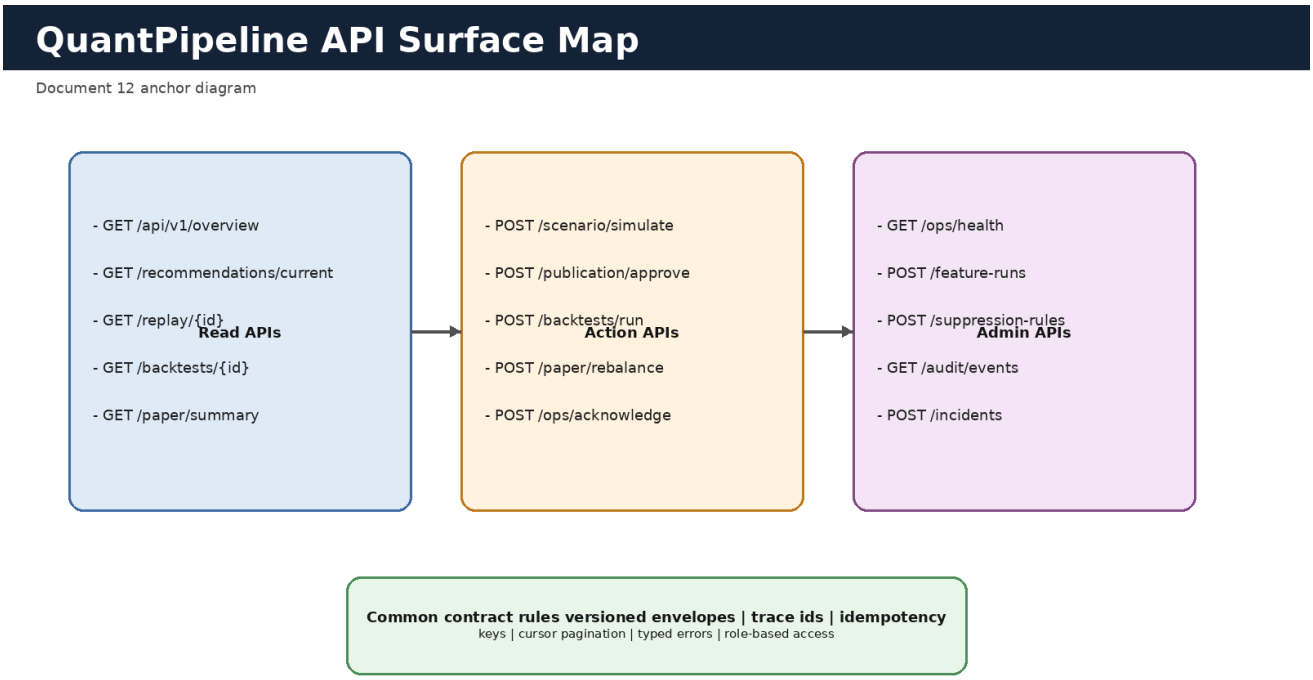

Figure 12-1. High-level API surface map.

3. Contract Principles

- Version every public route namespace.
- Return typed envelopes with metadata, data payload, and warnings.
- Expose trace_id and recommendation or run references where relevant.
- Use typed error objects instead of free-form error strings.
- Keep user-facing semantics stable even if internal services change.
- Mark temporary scenario outputs explicitly so they cannot be confused with published recommendations.
- Support partial and degraded states through explicit status fields rather than transport-level ambiguity.
- Design for desktop-first payload richness while staying safe for mobile clients.

4. Authentication and Authorization

The initial product is private, but the API still requires explicit authentication and role-aware authorization. At minimum, distinguish between read, simulate, approve/publish, and admin/ops actions. This avoids hard-wiring a security exception that would later weaken auditability.

Recommended mechanism: secure session or token-based auth at the gateway, server-side session validation, and policy checks in command handlers. Every mutation endpoint should emit an audit event containing actor, action, object reference, and outcome.

Role	Read recommendation	Run scenario	Publish/ approve	Admin mutate
Viewer	Yes	Optional	No	No
Operator	Yes	Yes	Conditional	Limited
Admin	Yes	Yes	Yes	Yes
Service account	Scoped	Scoped	No human publication	Scoped

5. Common Envelope and Metadata

All successful responses should follow a common envelope so that clients can handle freshness, warnings, and traceability consistently.

{ "meta": { "trace_id": "...", "api_version": "v1", "generated_at": "...", "warnings": [], "freshness": {"state": "healthy", "age_sec": 120} }, "data": { ... } }

For collections, add pagination metadata. For asynchronous operations, return a job resource reference. For degraded states, use warnings and status fields inside data rather than silently omitting relevant context.

6. Standard Types

The following object families recur across the API surface and should retain stable names: RecommendationSummary, RecommendationDetail, DecisionStageDetail, EngineComparisonRow, ReplayNarrative, BacktestSummary, PaperPortfolioSummary, AlertSummary, IncidentDetail, ApprovalDecision, and TypedError.

Where payloads differ by screen, preserve a shared core subset so clients can reuse rendering and avoid hidden semantic drift.

Type	Core fields
RecommendationSummary	recommendation_id, as_of, status, overall_confidence, top_changes, target_weights[]
DecisionStageDetail	stage_name, run_id, version_ref, summary, adjustments[], warnings[]
EngineComparisonRow	engine_id, stance, confidence, freshness_state, contribution_summary
ReplayNarrative	replay_id, recommendation_id, chronology[], stage_diffs[], evidence_refs[]
TypedError	code, category, message, retryable, object_ref, trace_id

7. Reference and Setup Endpoints

These endpoints support the initial shell, filters, universes, watchlists, and settings selectors used across the product.

Method	Path	Purpose	Notes
GET	/api/v1/reference/universes	List available universes	Supports status and scope filters
GET	/api/v1/reference/watchlists	List user watchlists	May include counts and freshness
GET	/api/v1/reference/benchmarks	List benchmarks	For backtests and paper
GET	/api/v1/reference/assets/search	Search instruments	Supports ticker and name query
GET	/api/v1/settings/current	Load saved user settings	Includes preferred horizon and display rules
POST	/api/v1/settings/current	Update user settings	Audit required for changed defaults

8. Overview and Current Recommendation Endpoints

The overview surface answers the question: what is the recommendation right now, how healthy is it, and what changed? These routes must be fast and read-model oriented.

Method	Path	Response	Behavior
GET	/api/v1/overview	OverviewSnapshot	Primary home screen payload
GET	/api/v1/recommendations/current	RecommendationDetail	Returns the latest published or staged recommendation for the selected context
GET	/api/v1/recommendations/{id}	RecommendationDetail	Historical detail by stable id
GET	/api/v1/recommendations/{id}/diff	RecommendationDiff	Changes vs previous recommendation
GET	/api/v1/recommendations/{id}/weights	RecommendationWeights	Portfolio table plus change metrics

RecommendationDetail should include the confidence decomposition, freshness state, top drivers, top warnings, links to decision-stage refs, and publication metadata so the client does not need to make multiple calls for a normal detail view.

9. Decision Breakdown Endpoints

These endpoints power the main dashboard and drill-down experiences for selection, allocation, timing, and risk overlay.

Method	Path	Purpose	Key query params
GET	/api/v1/	Compact breakdown	stage_filter,

Method	Path	Purpose	Key query params
	recommendations/{id}/decision-breakdown	across all stages	include_rationales
GET	/api/v1/selection-runs/{id}	Selection detail and shortlist	include_exclusions
GET	/api/v1/allocation-runs/{id}	Pre-risk allocation detail	include_contributors
GET	/api/v1/timing-runs/{id}	Timing adjustments and posture	include_rule_triggers
GET	/api/v1/risk-runs/{id}	Risk overlay actions and caps	include_before_after

10. Engine Comparison Endpoints

The comparison surface should help the user understand agreement, conflict, and evidence asymmetry without exposing raw internal artifacts by default.

Method	Path	Response	Notes
GET	/api/v1/recommendations/{id}/engines	EngineComparisonList	Matrix-oriented summary for the current recommendation
GET	/api/v1/engine-runs/{id}	EngineRunDetail	Deep detail for one engine run
GET	/api/v1/engines/status	EngineStatusList	Freshness, last success, and promoted version
GET	/api/v1/engines/{engine_id}/history	EngineRunHistory	Recent history for diagnostics

11. Scenario Simulation Endpoints

Scenario endpoints allow the user or operator to explore what-if changes without publishing them as the current recommendation. Every response must state clearly that the output is simulated and temporary.

Method	Path	Purpose	Required request fields
POST	/api/v1/scenarios/simulate	Run temporary scenario	context, scenario_overrides, requested_stages
GET	/api/v1/scenarios/{id}	Fetch stored scenario result	none
POST	/api/v1/scenarios/{id}/pin	Persist scenario preset	name, notes

A scenario result should include scenario_id, base_recommendation_id if any, simulated stage outputs, simulated target weights, deltas vs current recommendation, warnings, and an explicit `is_published = false` marker.

12. Replay and Forensics Endpoints

Replay endpoints should reconstruct one historical recommendation end-to-end. They are read-only but high value for trust and learning.

Method	Path	Purpose
GET	/api/v1/replays/{recommendation_id}	Open or create replay narrative for a historical recommendation
GET	/api/v1/replays/{replay_id}/stages/{stage_name}	Stage-specific replay drill-down
GET	/api/v1/replays/{replay_id}/evidence	Evidence summary with source references and confidence
GET	/api/v1/replays/{replay_id}/changes	What changed vs prior recommendation

13. Backtest and Validation Endpoints

Validation endpoints are primarily admin or operator scoped. They must support job initiation, result retrieval, artifact listing, and promotion decisions.

Method	Path	Purpose	Notes
POST	/api/v1/backtests/run	Start backtest job	Requires methodology and policy bundle references
GET	/api/v1/backtests/{id}	Read backtest summary	High-level metrics and run status
GET	/api/v1/backtests/{id}/periods	Read period-level results	Supports pagination
GET	/api/v1/backtests/{id}/artifacts	List generated artifacts	Charts, reports, raw metrics exports
POST	/api/v1/backtests/{id}/promote-check	Evaluate promotion gate	Returns pass/fail and failed rules

14. Paper Portfolio Endpoints

Paper endpoints track shadow performance and operational drift between the policy and simulated holding evolution.

Method	Path	Purpose
GET	/api/v1/paper/summary	Paper portfolio home payload
GET	/api/v1/paper/rebalances	Historical rebalances and

Method	Path	Purpose
		reasons
GET	/api/v1/paper/holdings	Current paper holdings
GET	/api/v1/paper/drift	Drift and benchmark-relative state
POST	/api/v1/paper/rebalance-now	Operator-triggered paper refresh

15. Admin and Ops Endpoints

Admin endpoints provide the health and control plane needed to keep recommendation publication honest.

Method	Path	Purpose	Privilege
GET	/api/v1/ops/health	System health summary	Operator
GET	/api/v1/ops/alerts	List active and historical alerts	Operator
POST	/api/v1/ops/alerts/{id}/acknowledge	Acknowledge alert	Operator
GET	/api/v1/ops/incidents/{id}	Incident detail	Operator
POST	/api/v1/ops/incidents	Create incident	Operator
POST	/api/v1/publication/{id}/approve	Approve publication	Admin
POST	/api/v1/publication/{id}/suppress	Suppress publication	Admin
GET	/api/v1/audit/events	Query audit events	Admin
POST	/api/v1/policies/promote	Promote policy bundle	Admin

16. Error Model

Errors should be typed and actionable. Clients must know whether an error is retryable, whether it indicates a degraded but still usable state, and whether it blocks publication.

Recommended categories: validation_error, authorization_error, not_found, conflict, dependency_failure, degraded_state, and internal_error.

{ "error": { "code": "DEPENDENCY_STALE", "category": "degraded_state", "message": "News feed freshness exceeded publication threshold", "retryable": false, "object_ref": "source:news", "trace_id": "..." } }

17. Idempotency, Concurrency, and Pagination

- Use idempotency keys for scenario creation, approval decisions, and backtest run starts when duplicate submission is possible.

- Protect publication and suppression actions with optimistic concurrency tokens tied to the publication record state.
- Use cursor pagination for growing collections such as alerts, audit events, article lists, and historical run history.
- Return explicit sort keys in paginated collections so replayable views stay stable.

18. Versioning and Deprecation

The route namespace should begin with `/api/v1`. Breaking response changes require a new version. Additive fields are allowed within a version as long as required fields remain stable and semantic meaning does not change.

For high-value types such as RecommendationDetail and DecisionStageDetail, publish compatibility notes before removing or renaming fields. The platform should prefer deprecation windows over abrupt contract changes because replay and historical comparisons depend on contract continuity.

19. Contract Acceptance Checklist

1. Every route has a typed success envelope and typed error format.
2. Every user-visible recommendation response carries freshness, status, and trace metadata.
3. Every mutation route emits an audit event.
4. Scenario outputs cannot be mistaken for published recommendations.
5. Replay, backtests, paper, and admin routes resolve stable identifiers from Document 11.
6. Authorization behavior is explicit and testable for every mutation route.

Appendix A - Example Recommendation Detail Skeleton

{ "meta": {"trace_id": "...", "api_version": "v1", "generated_at": "..."}, "data": { "recommendation_id": "...", "status": {"publication_state": "published", "health_state": "healthy"}, "confidence": {"overall": 0.72, "data": 0.81, "model": 0.69, "operational": 0.93}, "target_weights": [{"asset_id": "AAPL", "target_weight": 0.08}], "decision_breakdown_ref": {"selection_run_id": "...", "allocation_run_id": "...", "timing_run_id": "...", "risk_run_id": "..."}, "top_drivers": ["macro improved", "trend support"], "warnings": [] } }

Appendix B - Example Approval Command

POST /api/v1/publication/{id}/approve

{ "expected_state": "staged_for_approval", "decision_note": "Approved after verifying source freshness and no active incident", "idempotency_key": "..." }